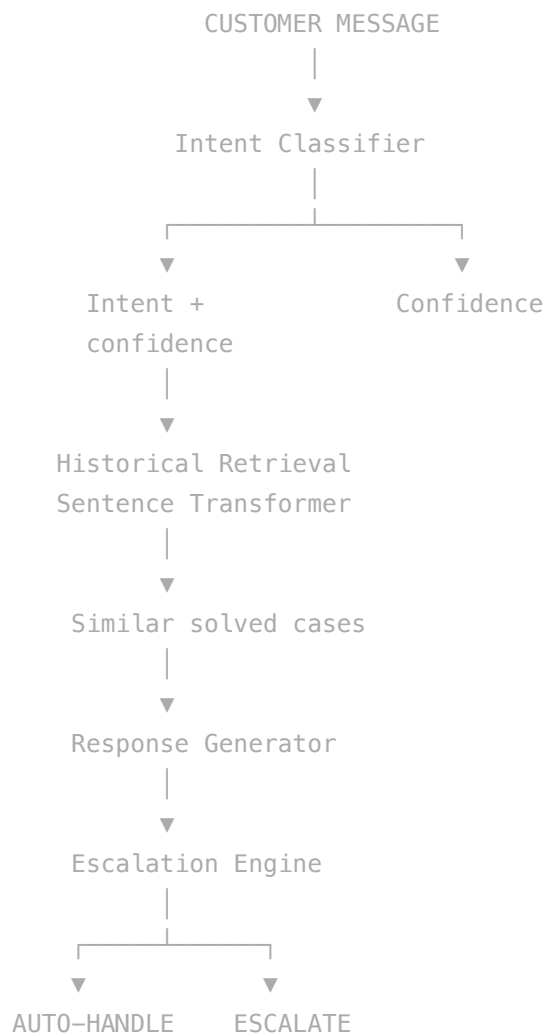


Yes. We can build this as a **complete end-to-end project**, including the dataset analysis, intent classifier, historical-response retrieval, escalation engine, evaluation, and Streamlit UI.

One important point first: the dataset is a CSV where each row is a tweet, with `tweet_id` , `author_id` , `inbound` , `text` , `response_tweet_id` , and `in_response_to_tweet_id` . The conversation links let us reconstruct customer → brand interactions. [k Kaggle](#) +1

I recommend **not using an LLM for everything**. For an internship evaluation, a hybrid system is easier to explain and evaluate:



Sentence Transformers is well suited for semantic retrieval because it converts text into embeddings that can be compared for semantic similarity. [GitHub](#) +1

1. What we are going to build

Project name:

AI Customer Support Agent

Features:

Input

"My order hasn't arrived yet. Can you check?"

Output

Intent:
Delivery Issue

Confidence:
0.91

Similar historical cases:
3

Draft reply:
"Sorry about the delay. Please DM us your order number so we can check the delivery status for you."

Decision:
AUTO-HANDLE

Reason:
High-confidence intent with strong historical precedent.

For a risky message:

"I've been charged twice and nobody has fixed it."

Output could be:

Intent:
Billing Issue

Confidence:
0.87

Decision:
ESCALATE

Reason:
Repeated unresolved billing issue requires account-specific investigation.

2. Project architecture

Create this folder:

```
customer-support-ai-agent/  
|  
├─ data/  
|   └─ raw/
```

```
| | | └─ customer_support.csv
| | |
| | └─ processed/
| |   ├── brand_tweets.csv
| |   ├── support_pairs.csv
| |   └─ labeled_data.csv
| |
| └─ models/
|   ├── intent_classifier.pkl
|   ├── intent_vectorizer.pkl
|   └─ embeddings.npy
|
| └─ config/
|   └─ intents.json
|
| └─ src/
|   ├── __init__.py
|   ├── config.py
|   ├── data_processing.py
|   ├── brand_selection.py
|   ├── intent_classifier.py
|   ├── retrieval.py
|   ├── response_generator.py
|   ├── escalation.py
|   └─ agent.py
|
| └─ scripts/
|   ├── 01_prepare_data.py
|   ├── 02_select_brand.py
|   ├── 03_create_intents.py
|   ├── 04_train_classifier.py
|   └─ 05_build_retriever.py
|
| └─ evaluation/
|   └─ evaluate.py
|
| └─ app.py
|
| └─ requirements.txt
| └─ README.md
| └─ .gitignore
```

3. Download the dataset

The official Kaggle dataset is:

[Customer Support on Twitter — Kaggle](#)

It contains more than 3 million tweets/replies. [k](#) Kaggle

After downloading it, put the CSV here:

```
customer-support-ai-agent/data/raw/customer_support.csv
```

The filename doesn't actually matter; we'll configure it.

4. Create the project on your Mac

Open Terminal:

```
cd ~/Desktop
```

Create the project:

```
mkdir customer-support-ai-agent  
cd customer-support-ai-agent
```

Create folders:

```
mkdir -p data/raw  
mkdir -p data/processed  
mkdir -p models  
mkdir -p config  
mkdir -p src  
mkdir -p scripts  
mkdir -p evaluation
```

5. Create Python virtual environment

I recommend Python **3.10 or newer** for this project because current Sentence Transformers documentation recommends Python 3.10+.  [GitHub](#)

Check:

```
python3 --version
```

If you have Python 3.10+:

```
python3 -m venv .venv
```

Activate:

```
source .venv/bin/activate
```

You should see:

```
(.venv)
```

at the beginning of your terminal.

6. requirements.txt

Create:

```
requirements.txt
```

Put:

```
pandas  
numpy  
scikit-learn  
sentence-transformers  
streamlit  
joblib  
matplotlib  
seaborn  
plotly
```

Install:

```
pip install -r requirements.txt
```

The Sentence Transformers package supports embedding generation and semantic search/retrieval.  [GitHub](#) +1

7. First important step — inspect the dataset

Create:

```
scripts/01_prepare_data.py
```

Use:

```
import pandas as pd
import os

INPUT_FILE = "data/raw/customer_support.csv"

OUTPUT_DIR = "data/processed"

os.makedirs(OUTPUT_DIR, exist_ok=True)

print("Loading dataset...")

df = pd.read_csv(INPUT_FILE)

print("\nDataset shape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
print(df.head())

print("\nMissing values:")
print(df.isnull().sum())

print("\nInbound distribution:")
print(df["inbound"].value_counts())

print("\nNumber of unique authors:")
print(df["author_id"].nunique())

print("\nNumber of unique tweets:")
print(df["tweet_id"].nunique())
```

Run:

```
python scripts/01_prepare_data.py
```

You should get something similar to:

```
Dataset shape:
(3000000+, 7)

Columns:
[
  'tweet_id',
  'author_id',
  'inbound',
  'created_at',
  'text',
  'response_tweet_id',
  'in_response_to_tweet_id'
]
```

The exact number can differ depending on the dataset version.

8. Understand inbound

This is extremely important for explaining your project.

According to the dataset documentation:

```
inbound = True
```

means the tweet is coming **towards the support company**.

Therefore:

```
inbound = True
      ↓
Customer message
```

And:

```
inbound = False
      ↓
Brand/support response
```

The dataset specifically provides `in_response_to_tweet_id` and `response_tweet_id` so conversations can be reconstructed. [k](#) [Kaggle](#)

9. Find the best brand/company

The dataset anonymizes Twitter IDs, so you shouldn't invent a brand name if your copy of the dataset doesn't contain one.

Instead, we'll identify the support account by its response activity.

Create:

```
scripts/02_select_brand.py
```

```
import pandas as pd

INPUT_FILE = "data/raw/customer_support.csv"

df = pd.read_csv(INPUT_FILE)

# Brand/support tweets
outbound = df[df["inbound"] == False].copy()

print("Number of outbound/support tweets:", len(outbound))

brand_counts = (
    outbound
    .groupby("author_id")
    .size()
    .sort_values(ascending=False)
)

print("\nTop support accounts:\n")
print(brand_counts.head(20))
```

Run:

```
python scripts/02_select_brand.py
```

You'll get:

```
Top support accounts:
```

```
author_id
xxxxx    120000
xxxxx    98000
```

```
xxxxx      87000
...
```

For the first working version, choose a support account with a **large number of conversations but not an overwhelmingly huge amount of data**.

For example:

```
BRAND_ID = "xxxxx"
```

10. Extract conversations for our selected brand

Now create:

```
src/data_processing.py
```

```
import pandas as pd
```

```
def load_data(path):
    return pd.read_csv(path)
```

```
def clean_text(text):
    if pd.isna(text):
        return ""

    text = str(text)
    text = text.replace("\n", " ")
    text = " ".join(text.split())

    return text
```

```
def create_tweet_lookup(df):
    return {
        str(row["tweet_id"]): row
        for _, row in df.iterrows()
    }
```

```
def extract_support_pairs(df, brand_id):

    tweet_lookup = create_tweet_lookup(df)
```

```

pairs = []

brand_tweets = df[
    (df["author_id"] == brand_id) &
    (df["inbound"] == False)
]

for _, brand_tweet in brand_tweets.iterrows():

    parent_id = brand_tweet["in_response_to_tweet_id"]

    if pd.isna(parent_id):
        continue

    parent_id = str(parent_id)

    if parent_id not in tweet_lookup:
        continue

    customer_tweet = tweet_lookup[parent_id]

    if customer_tweet["inbound"] != True:
        continue

    customer_text = clean_text(customer_tweet["text"])
    brand_text = clean_text(brand_tweet["text"])

    if not customer_text or not brand_text:
        continue

    pairs.append({
        "customer_tweet_id": customer_tweet["tweet_id"],
        "brand_tweet_id": brand_tweet["tweet_id"],
        "customer_text": customer_text,
        "brand_response": brand_text
    })

return pd.DataFrame(pairs)

```

11. Create the extraction script

Create:

scripts/03_extract_brand.py

```

import pandas as pd
import sys
import os

sys.path.append(".")

from src.data_processing import load_data, extract_support_pairs

INPUT_FILE = "data/raw/customer_support.csv"

# CHANGE THIS AFTER RUNNING 02_select_brand.py
BRAND_ID = "PUT_SELECTED_BRAND_ID_HERE"

OUTPUT_FILE = "data/processed/support_pairs.csv"

print("Loading dataset...")

df = load_data(INPUT_FILE)

print("Extracting conversations...")

pairs = extract_support_pairs(
    df,
    BRAND_ID
)

print("\nNumber of support pairs:")
print(len(pairs))

os.makedirs("data/processed", exist_ok=True)

pairs.to_csv(
    OUTPUT_FILE,
    index=False
)

print("\nSaved:")
print(OUTPUT_FILE)

```

Run:

```
python scripts/03_extract_brand.py
```

Now you have:

```
customer message
```

↓

historical brand response

This becomes the foundation of the entire project.

12. Now define the intents

This part should **not be completely automated**.

The assignment explicitly says:

intents that you define from the data

So inspect your extracted conversations.

Run:

```
import pandas as pd

df = pd.read_csv(
    "data/processed/support_pairs.csv"
)

pd.set_option("display.max_colwidth", 200)

print(
    df[
        ["customer_text", "brand_response"]
    ].sample(100, random_state=42).to_string(index=False)
)
```

Look for recurring problems.

For example, you might discover:

```
Delivery
Payment
Refund
Account
Technical Issue
Cancellation
Product Issue
Complaint
General Question
Other
```

But your final intent names should come from your selected brand's actual conversations.

13. Create intents.json

For example:

config/intents.json

```
{
  "delivery_issue": {
    "description": "Problems involving delayed, missing, or incorrect deliveries"
    "examples": [
      "where is my order",
      "my package hasn't arrived",
      "delivery is delayed"
    ]
  },
  "payment_issue": {
    "description": "Problems involving payment, charges, or billing",
    "examples": [
      "I was charged twice",
      "payment failed",
      "wrong amount charged"
    ]
  },
  "refund_request": {
    "description": "Requests concerning refunds or returning money",
    "examples": [
      "I want a refund",
      "when will I receive my refund",
      "please refund my order"
    ]
  },
  "account_issue": {
    "description": "Problems involving login, account access, or account settings",
    "examples": [
      "I can't login",
      "my account is locked",
      "I cannot access my account"
    ]
  },
}
```

```

"technical_issue": {
  "description": "Technical problems with the product or service",
  "examples": [
    "the app isn't working",
    "website is broken",
    "service stopped working"
  ]
},

"cancellation": {
  "description": "Requests to cancel an order, service, or subscription",
  "examples": [
    "cancel my order",
    "I want to cancel",
    "please cancel this"
  ]
},

"complaint": {
  "description": "Customer dissatisfaction or unresolved complaints",
  "examples": [
    "this is unacceptable",
    "very disappointed",
    "terrible service"
  ]
},

"general_question": {
  "description": "General information requests",
  "examples": [
    "how does this work",
    "what are your hours",
    "can you help me"
  ]
}
}

```

Do not copy these blindly.

After we inspect your actual brand, we'll customize them.

14. Build training labels

Because the raw dataset doesn't provide intent labels, we need to label historical customer messages.

For a strong internship project, I recommend starting with **manual labeling of around 1,000–3,000 messages**.

Create:

```
data/processed/labeled_data.csv
```

with:

```
customer_text,intent
"where is my order",delivery_issue
"my payment failed",payment_issue
"please refund me",refund_request
"I can't login",account_issue
```

This is actually a positive point in your project:

"Intent taxonomy was derived from the selected brand's historical support conversations and manually validated."

15. Train the intent classifier

We can start with TF-IDF + Logistic Regression.

This is fast, interpretable, and gives us probabilities useful for escalation.

Create:

```
src/intent_classifier.py
```

```
import joblib

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.pipeline import Pipeline

from sklearn.model_selection import train_test_split

from sklearn.metrics import (
    classification_report,
    confusion_matrix
)

def train_classifier(df):
```



```

X = df["customer_text"]
y = df["intent"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            lowercase=True,
            ngram_range=(1, 2),
            min_df=2,
            max_features=50000,
            sublinear_tf=True
        )
    ),
    (
        "classifier",
        LogisticRegression(
            max_iter=2000,
            class_weight="balanced"
        )
    )
])

```

```

model.fit(X_train, y_train)

```

```

predictions = model.predict(X_test)

```

```

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        predictions
    )
)

```

```

print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        predictions
    )
)

```

```
return model
```

16. Train it

Create:

```
scripts/04_train_classifier.py
```

```
import pandas as pd
import joblib
import os
import sys

sys.path.append(".")

from src.intent_classifier import train_classifier

DATA_FILE = "data/processed/labeled_data.csv"

MODEL_FILE = "models/intent_classifier.pkl"

df = pd.read_csv(DATA_FILE)

print("Dataset:")
print(df.shape)

print("\nIntent distribution:")
print(df["intent"].value_counts())

model = train_classifier(df)

os.makedirs("models", exist_ok=True)

joblib.dump(
    model,
    MODEL_FILE
)

print("\nModel saved:")
print(MODEL_FILE)
```

Run:

```
python scripts/04_train_classifier.py
```

17. Why Logistic Regression?

This is actually a good interview talking point.

You can explain:

```
Tweet
↓
TF-IDF
↓
Numerical feature vector
↓
Logistic Regression
↓
Intent probabilities
```

Example:

delivery_issue	0.91
payment_issue	0.04
refund_request	0.03
account_issue	0.01
technical_issue	0.01

Therefore:

```
Predicted intent = delivery_issue
Confidence = 0.91
```

18. Build semantic retrieval

Now comes the most important AI part.

We need to retrieve historical conversations that are semantically similar.

We'll use:

```
sentence-transformers/all-MiniLM-L6-v2
```

This is a lightweight embedding model and Sentence Transformers explicitly supports semantic search using embeddings.  [GitHub](#) +1

19. Retrieval code

Create:

```
src/retrieval.py
```

```
import numpy as np

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity

class HistoricalRetriever:

    def __init__(self, dataframe):

        self.df = dataframe.reset_index(drop=True)

        self.model = SentenceTransformer(
            "sentence-transformers/all-MiniLM-L6-v2"
        )

        print("Creating embeddings...")

        self.embeddings = self.model.encode(
            self.df["customer_text"].tolist(),
            normalize_embeddings=True,
            show_progress_bar=True
        )

    def search(
        self,
        query,
        top_k=5
    ):

        query_embedding = self.model.encode(
            [query],
            normalize_embeddings=True
        )

        similarities = cosine_similarity(
```

```

        query_embedding,
        self.embeddings
    )[0]

    indices = np.argsort(
        similarities
    )[:, :-1][:top_k]

    results = []

    for index in indices:

        result = self.df.iloc[index].to_dict()

        result["similarity"] = float(
            similarities[index]
        )

        results.append(result)

    return results

```

20. Test retrieval

Create:

scripts/05_test_retrieval.py

```

import pandas as pd
import sys

sys.path.append(".")

from src.retrieval import HistoricalRetriever

df = pd.read_csv(
    "data/processed/support_pairs.csv"
)

retriever = HistoricalRetriever(df)

query = input(
    "\nEnter customer message: "
)

```

```

results = retriever.search(
    query,
    top_k=5
)

print("\nSimilar historical cases:\n")

for i, result in enumerate(results):

    print("=" * 70)

    print(
        f"Similarity: "
        f"{result['similarity']:.3f}"
    )

    print(
        "Customer:",
        result["customer_text"]
    )

    print(
        "Brand:",
        result["brand_response"]
    )

```

Run:

```
python scripts/05_test_retrieval.py
```

Example:

```

Enter customer message:
My package is late

```

Output:

```

Similarity: 0.86

Customer:
my package still hasn't arrived

Brand:
Sorry about the delay. Please DM us your order number...

```

That is the historical grounding mechanism.

21. Response generation

For the first working version, **don't let an LLM invent company policies.**

Instead, use historical responses as the source of truth.

Create:

```
src/response_generator.py
```

```
import re

def clean_response(text):

    text = str(text)

    text = re.sub(
        r"\s+",
        " ",
        text
    )

    return text.strip()

def generate_response(
    customer_message,
    retrieved_cases,
    threshold=0.55
):

    if not retrieved_cases:
        return None

    best_case = retrieved_cases[0]

    similarity = best_case["similarity"]

    if similarity < threshold:
        return None

    response = clean_response(
        best_case["brand_response"]
    )

    return response
```

This gives us a very safe baseline:

```
New customer
  ↓
Find similar historical customer
  ↓
Use historical brand response
```

22. But we can improve this

Once the basic system works, we can add an LLM.

The architecture becomes:

```
Customer message
  ↓
Intent
  ↓
Historical examples
  ↓
LLM
  ↓
Grounded response
```

Prompt:

You are a customer support agent.

You must answer the customer using ONLY the historical support examples provided below.

Do not invent:

- policies
- refunds
- prices
- dates
- account information

Historical examples:

...

Customer message:

...

Write a concise professional support response.

This makes your project significantly stronger, but I recommend getting the **non-LLM baseline** working first.

23. Build escalation engine

Create:

src/escalation.py

```
def escalation_decision(
    intent_confidence,
    retrieval_similarity,
    customer_message,
    intent
):

    message = customer_message.lower()

    reasons = []

    # 1. Low intent confidence
    if intent_confidence < 0.60:

        reasons.append(
            "Low intent classification confidence"
        )

    # 2. Weak historical match
    if retrieval_similarity < 0.55:

        reasons.append(
            "No strong historical precedent"
        )

    # 3. Account-specific signals
    account_terms = [
        "account",
        "charged",
        "payment",
        "refund",
        "money",
        "bank",
        "transaction"
```

```

]

if any(
    word in message
    for word in account_terms
):

    if intent in [
        "payment_issue",
        "refund_request",
        "account_issue"
    ]:

        reasons.append(
            "Account-specific issue may require human investigation"
        )

# 4. Legal/risk signals
risk_terms = [
    "lawyer",
    "legal",
    "police",
    "fraud",
    "lawsuit",
    "court"
]

if any(
    word in message
    for word in risk_terms
):

    reasons.append(
        "Potential high-risk or legal issue"
    )

# 5. Strong negative escalation
escalation_terms = [
    "still not fixed",
    "again",
    "third time",
    "nobody helped",
    "already contacted"
]

if any(
    phrase in message
    for phrase in escalation_terms
):

    reasons.append(
        "Customer reports repeated or unresolved issue"
    )

```

```

    )

    if reasons:

        return {
            "decision": "ESCALATE",
            "reason": "; ".join(reasons)
        }

    return {
        "decision": "AUTO-HANDLE",
        "reason": (
            "High-confidence classification with "
            "sufficient historical precedent and "
            "no major escalation signals."
        )
    }
}

```

24. Build the complete agent

Create:

src/agent.py

```

import joblib

from src.retrieval import HistoricalRetriever

from src.response_generator import (
    generate_response
)

from src.escalation import (
    escalation_decision
)

class CustomerSupportAgent:

    def __init__(
        self,
        classifier,
        historical_data
    ):

        self.classifier = classifier

```

```

        self.retriever = HistoricalRetriever(
            historical_data
        )

    def predict_intent(
        self,
        message
    ):

        prediction = self.classifier.predict(
            [message]
        )[0]

        probabilities = (
            self.classifier.predict_proba(
                [message]
            )[0]
        )

        confidence = max(
            probabilities
        )

        return prediction, confidence

    def process(
        self,
        message
    ):

        intent, confidence = (
            self.predict_intent(message)
        )

        historical_cases = (
            self.retriever.search(
                message,
                top_k=5
            )
        )

        best_similarity = 0

        if historical_cases:

            best_similarity = (
                historical_cases[0]["similarity"]
            )

        response = generate_response(
            message,

```

```

        historical_cases
    )

    decision = escalation_decision(
        confidence,
        best_similarity,
        message,
        intent
    )

    return {
        "message": message,
        "intent": intent,
        "intent_confidence": confidence,
        "historical_cases": historical_cases,
        "draft_response": response,
        "decision": decision["decision"],
        "decision_reason": decision["reason"]
    }

```

25. Streamlit application

Now create:

app.py

```

import streamlit as st
import pandas as pd
import joblib
import sys

sys.path.append(".")

from src.agent import CustomerSupportAgent

st.set_page_config(
    page_title="AI Customer Support Agent",
    page_icon="🤖",
    layout="wide"
)

@st.cache_resource
def load_agent():

```

```

classifier = joblib.load(
    "models/intent_classifier.pkl"
)

data = pd.read_csv(
    "data/processed/support_pairs.csv"
)

agent = CustomerSupportAgent(
    classifier,
    data
)

return agent

st.title(
    "🤖 AI Customer Support Agent"
)

st.write(
    """
    AI-powered customer support agent using
    historical brand conversations.
    """
)

agent = load_agent()

message = st.text_area(
    "Customer message",
    placeholder=(
        "Example: My order hasn't arrived yet..."
    ),
    height=150
)

if st.button(
    "Analyze Message",
    type="primary"
):

    if not message.strip():

        st.warning(
            "Please enter a customer message."
        )

    else:

        with st.spinner(

```

```

        "Analyzing..."
    ):

        result = agent.process(
            message
        )

    st.divider()

    col1, col2 = st.columns(2)

    with col1:

        st.subheader(
            "Intent"
        )

        st.success(
            result["intent"]
        )

    with col2:

        st.subheader(
            "Confidence"
        )

        st.metric(
            "Classification confidence",
            f"{result['intent_confidence']:.2%}"
        )

    st.divider()

    st.subheader(
        "Historical Cases"
    )

    for i, case in enumerate(
        result["historical_cases"],
        start=1
    ):

        with st.expander(
            f"Case {i} "
            f"(similarity: "
            f"{case['similarity']:.2%})"
        ):

            st.write(
                "**Customer:**"
            )

```

```

        st.write(
            case["customer_text"]
        )

        st.write(
            "**Historical Brand Response:**"
        )

        st.write(
            case["brand_response"]
        )

    st.divider()

    st.subheader(
        "Draft Response"
    )

    if result["draft_response"]:

        st.info(
            result["draft_response"]
        )

    else:

        st.warning(
            "No sufficiently similar historical "
            "response was found."
        )

    st.divider()

    st.subheader(
        "Decision"
    )

    if result["decision"] == "AUTO-HANDLE":

        st.success(
            "🟢 AUTO-HANDLE"
        )

    else:

        st.error(
            "🔴 ESCALATE TO HUMAN"
        )

    st.write(
        "**Reason:**",

```



```
        result["decision_reason"]
    )
```

26. Run the application

From the project root:

```
source .venv/bin/activate
```

Then:

```
streamlit run app.py
```

You should get something like:

```
Local URL:
http://localhost:8501
```

Open it in your browser.

27. What your complete agent is now doing

Suppose you enter:

```
My package is late. Where is it?
```

Internally:

```
"My package is late"
  |
  ▼
TF-IDF
  |
  ▼
Logistic Regression
  |
  ▼
delivery_issue
  |
  ▼
```



This is a complete AI pipeline.

28. But we are not finished

The assignment's most important sentence is:

"You then have to convince us the agent is good enough to trust."

So we need an **evaluation framework**.

29. Intent evaluation

Create:

evaluation/evaluate.py

```
import pandas as pd
import joblib

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score
```

```

)

DATA_FILE = (
    "data/processed/labeled_data.csv"
)

MODEL_FILE = (
    "models/intent_classifier.pkl"
)

df = pd.read_csv(
    DATA_FILE
)

model = joblib.load(
    MODEL_FILE
)

predictions = model.predict(
    df["customer_text"]
)

print("\nAccuracy:")
print(
    accuracy_score(
        df["intent"],
        predictions
    )
)

print("\nMacro F1:")
print(
    f1_score(
        df["intent"],
        predictions,
        average="macro"
    )
)

print("\nClassification Report:")

print(
    classification_report(
        df["intent"],
        predictions
    )
)

print("\nConfusion Matrix:")

```

```
print(
    confusion_matrix(
        df["intent"],
        predictions
    )
)
```

Run:

```
python evaluation/evaluate.py
```

30. We should also measure retrieval quality

For each test customer message:

```
Expected historical case
      ↓
Retriever
      ↓
Top 5 cases
```

Measure:

```
Recall@1
Recall@3
Recall@5
```

Example:

```
Recall@1 = 78%
Recall@3 = 89%
Recall@5 = 94%
```

That tells the interviewer:

"For 94% of test queries, a relevant historical support case appeared within the top five retrieved examples."

That's a much stronger argument than saying "we use RAG."

31. Evaluate escalation

Create a manually reviewed test set:

```
message,expected_decision
"where is my order",AUTO-HANDLE
"please tell me your support hours",AUTO-HANDLE
"I was charged twice",ESCALATE
"I've contacted you three times already",ESCALATE
"I will contact my lawyer",ESCALATE
```

Then compare:

```
Expected
vs
Predicted
```

Calculate:

```
Escalation precision
Escalation recall
Escalation F1
False auto-handle rate
False escalation rate
```

32. The most important metric

I would prominently show:

False Auto-Handle Rate

Example:

```
100 cases requiring human support
```

```
Agent incorrectly auto-handled:
4
```

```
False Auto-Handle Rate:
4%
```

Why?

Because this is a support system.

A wrong automated answer to a harmless FAQ is not as dangerous as an AI confidently responding to:

```
"Your system charged me twice."
```

or:

```
"This is fraud."
```

Therefore our design deliberately favors:

```
UNCERTAINTY  
↓  
ESCALATE
```

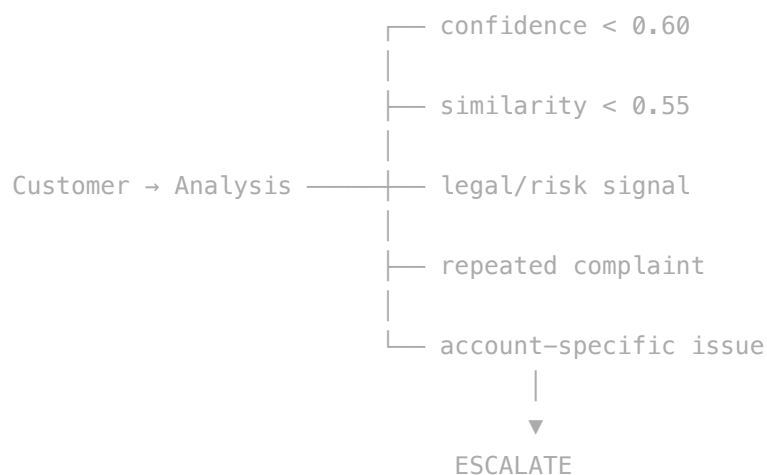
rather than:

```
UNCERTAINTY  
↓  
GUESS
```

That's a very strong point to explain in the interview.

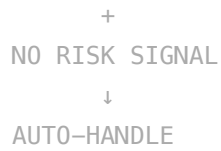
33. Add a confidence policy

Eventually I recommend this policy:



Otherwise:

```
HIGH CONFIDENCE  
+  
GOOD HISTORICAL MATCH
```

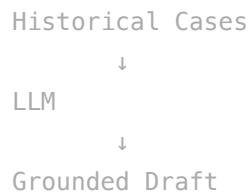


34. One improvement I strongly recommend

After the basic version works, replace the simple response selection:

```
best_case["brand_response"]
```

with:

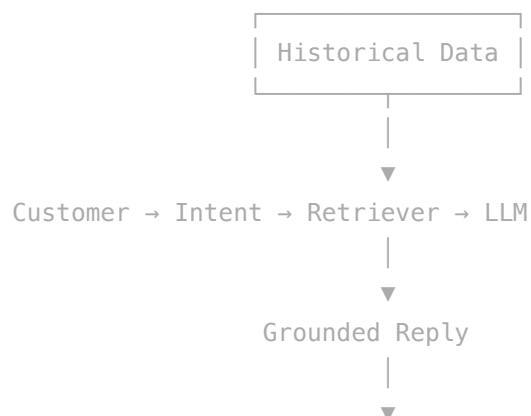


The LLM should receive only:

- Customer message
- Predicted intent
- Top 3–5 historical cases
- Historical brand responses

And the prompt should explicitly prohibit unsupported information.

That gives us:





35. What I want you to submit

Your final GitHub/project folder should contain:

```
customer-support-ai-agent/  
|  
├─ data/  
├─ models/  
├─ config/  
├─ src/  
├─ scripts/  
├─ evaluation/  
|  
├─ app.py  
├─ requirements.txt  
├─ README.md  
└─ report.pdf
```

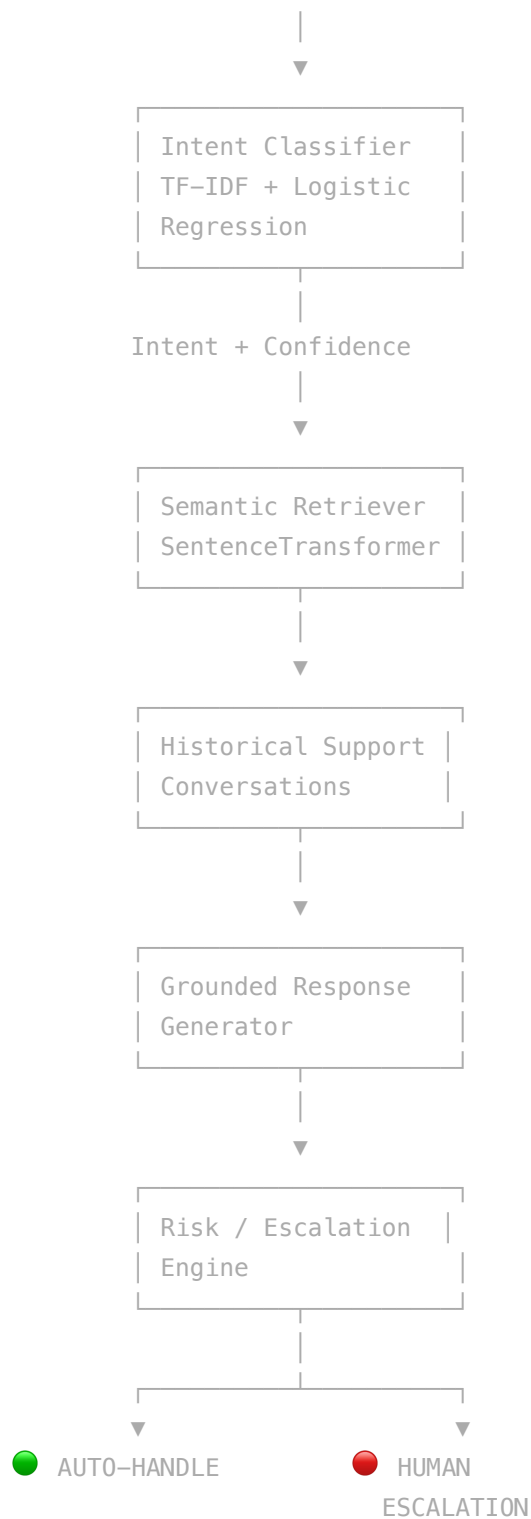
Your README should explain:

1. Problem
2. Dataset
3. Brand selection
4. Intent taxonomy
5. Intent classification
6. Historical retrieval
7. Response generation
8. Escalation strategy
9. Evaluation
10. Limitations
11. Future improvements
12. How to run

36. Your final architecture for the internship

I recommend we ultimately make the project:

Customer Message



This gives you **three separately measurable AI capabilities:**

Intent Classification

+

Historical Semantic Retrieval

+

Safe Decision Making

rather than just making a chatbot.

One thing I need from you before we make this truly "complete"

Upload the actual CSV you downloaded from Kaggle. The dataset structure is known, but the exact brand/support-account distribution in your copy determines which brand we should choose and which intents we should define. [k](#) Kaggle

 Upload a file

Once you provide it, we can proceed from **Stage 1 with your actual data**: identify the best brand automatically → inspect its conversations → derive the exact intents → create the labeled dataset → train the classifier → build retrieval → build the agent → evaluate it → finish the Streamlit application.



Today 8:17 AM

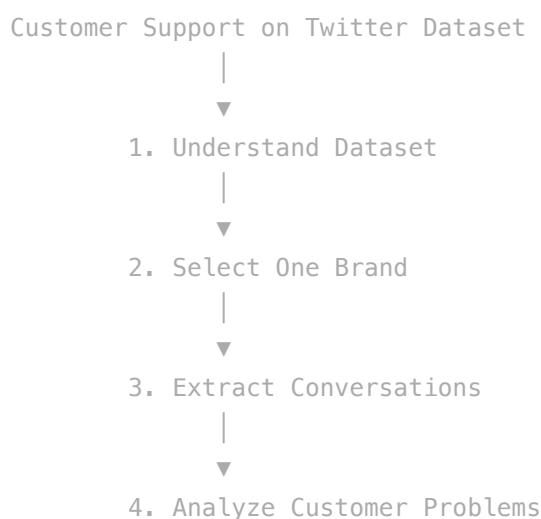
can you give me complete flow of project pointwise explained

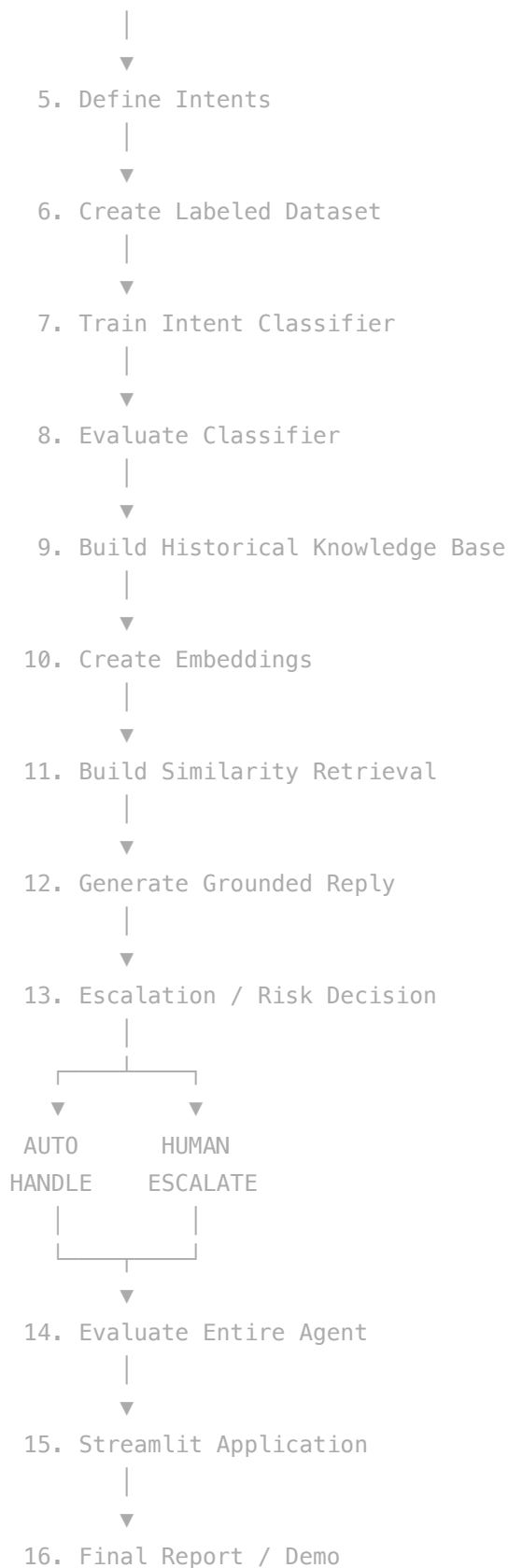
Yes. Think of the whole project as **one pipeline with 3 major AI jobs**:

1. **Understand the customer** → Intent Classification
2. **Know how the brand usually responds** → Historical Retrieval + Reply Generation
3. **Know when not to trust the AI** → Escalation Decision

Below is the complete flow point-by-point, including what happens to the data at every stage.

Complete Project Flow





1. Dataset Understanding

We start with the **Customer Support on Twitter** dataset.

It contains real customer-support interactions between customers and brands.

Important columns include:

```
tweet_id
author_id
inbound
created_at
text
response_tweet_id
in_response_to_tweet_id
```

What does each mean?

For example:

```
tweet_id = 123
author_id = ABC
inbound = True
text = "My order hasn't arrived"
```

`inbound=True` means the message is coming **toward the support account**, so we treat it as a customer message.

A response might look like:

```
tweet_id = 456
author_id = BRAND
inbound = False
text = "Please DM us your order number."
in_response_to_tweet_id = 123
```

The important relationship is:

```
Customer Tweet
|
| in_response_to_tweet_id
▼
Brand Response
```

Example

```
Customer:
"Where is my package?"
```

↓

Brand:

"Please DM us your order number and we'll check."

This relationship is extremely important because later we use these historical interactions to teach our agent **how the brand historically handled problems**.

2. Select ONE Brand

The assignment says:

Pick one brand from the dataset.

So we don't build an agent for every company.

First we calculate:

Brand A → 10,000 support conversations

Brand B → 50,000

Brand C → 5,000

Brand D → 80,000

Then select one suitable brand.

Selection criteria

We want a brand with:

- enough conversations
- enough customer messages
- enough brand responses
- different types of support problems
- good-quality conversations

Why?

Suppose:

Brand A

500 conversations

There isn't enough historical data.

But:

Brand B

50,000 conversations

gives us much more information.

3. Extract Conversations

Now we isolate only the selected brand.

Raw dataset:

```
3M+ tweets
  ↓
Selected brand
  ↓
Relevant customer tweets
  +
Brand responses
```

We create pairs:

```
customer_text
brand_response
```

Example:

Customer

Brand

Where is my order?

Please DM your order number

I can't login

Try resetting your password

I was charged twice

Please DM us so we can investigate

This becomes our **historical support knowledge base**.

4. Analyze Customer Problems

Now we study what customers are actually asking.

Suppose we inspect 1,000–3,000 messages.

We may discover patterns such as:

```
Delivery
Payment
Refund
```

Account
Technical Issue
Cancellation
Complaint
General Question

But this is **not decided beforehand**.

We derive the categories from the actual selected brand's conversations.

5. Define Intents

An **intent** means:

What is the customer trying to accomplish?

For example:

"My package hasn't arrived."

Intent:
DELIVERY_ISSUE

Another:

"I was charged twice."

Intent:
PAYMENT_ISSUE

Another:

"I want my money back."

Intent:
REFUND_REQUEST

So:

Customer message
↓
"What does the customer want?"
↓
Intent

Example

"My package hasn't arrived"

↓

DELIVERY_ISSUE

"I can't access my account"

↓

ACCOUNT_ISSUE

"Please cancel my order"

↓

CANCELLATION

6. Create a Labeled Dataset

The original dataset doesn't necessarily give us the exact intent labels we need.

Therefore, we create:

customer_text → intent

Example:

"My order is late"

→ delivery_issue

"Where is my package?"

→ delivery_issue

"I was charged twice"

→ payment_issue

"Payment failed"

→ payment_issue

"I need a refund"

→ refund_request

This becomes our supervised learning dataset.

For example:

labeled_data.csv

with:


```
customer_text,intent
"My order is late",delivery_issue
"Where is my package?",delivery_issue
"I was charged twice",payment_issue
"I need a refund",refund_request
```

7. Train Intent Classifier

Now we build the first AI component.

```
Customer Message
    ↓
Text preprocessing
    ↓
TF-IDF
    ↓
Logistic Regression
    ↓
Intent
```

Why TF-IDF?

Computers cannot directly understand:

```
"My package is late"
```

So we convert the text into numerical features.

TF-IDF identifies important words and phrases.

Example:

```
"My package is late"
    ↓
[0.12, 0.45, 0.00, 0.73, ...]
```

Then Logistic Regression learns relationships between these features and intents.

8. Classifier Gives Probability

This is very important because we need the confidence later.

Suppose customer says:

"My order hasn't arrived."

Classifier might output:

delivery_issue	0.92
payment_issue	0.03
refund_request	0.02
account_issue	0.01
technical_issue	0.02

Therefore:

Predicted intent:
delivery_issue

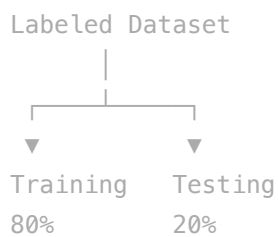
Confidence:
92%

This confidence will later help our escalation system.

9. Evaluate Intent Classifier

We don't simply train the model and assume it works.

We split:



Train on 80%.

Test on 20%.

Then calculate:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

For example:

Accuracy = 91%

Macro F1 = 89%

Why F1?

Because some intents may have fewer examples than others.

Example:

Delivery → 1,000 examples

Refund → 200 examples

Accuracy alone may hide poor performance on smaller categories.

10. Build Historical Knowledge Base

Now we move to the second major part.

We already have:

Customer → Brand response

historical pairs.

Example:

Customer:

"My package is late."

Brand:

"Sorry about the delay. Please DM your order number."

This is our historical knowledge.

We want the AI to be able to search this knowledge when a new customer asks something.

11. Create Text Embeddings

We use a Sentence Transformer such as:

all-MiniLM-L6-v2

It converts text into a vector.

For example:

```
"My package is late."
      ↓
[0.21, -0.14, 0.52, ...]
```

Another sentence:

```
"My order hasn't arrived."
      ↓
[0.23, -0.12, 0.49, ...]
```

These vectors should be close because the sentences have similar meanings.

12. Build Semantic Retrieval

Now suppose the new customer says:

```
"My delivery hasn't arrived."
```

We search historical conversations.

Instead of looking only for exact words, we calculate **semantic similarity**.

Example:

```
New message:
"My delivery hasn't arrived."
```

```
Historical Case 1:
"My package hasn't arrived."
Similarity = 0.91
```

```
Historical Case 2:
"Where is my order?"
Similarity = 0.87
```

```
Historical Case 3:
"I can't login."
Similarity = 0.21
```

Therefore:

Top historical cases:

1. 0.91
2. 0.87
3. 0.84

This is much better than simple keyword matching.

13. Why Historical Retrieval Matters

This is one of the strongest parts of the project.

Without retrieval:

```
Customer
  ↓
LLM
  ↓
Possibly invented answer
```

With retrieval:

```
Customer
  ↓
Historical conversations
  ↓
How the brand previously handled it
  ↓
Grounded answer
```

So the system follows the brand's historical behavior.

14. Generate the Reply

Now we have:

Customer:
"My delivery hasn't arrived."

Intent:
delivery_issue

Historical examples:

Case 1
Case 2
Case 3

We can generate a reply based on these examples.

For example:

"Sorry about the delay. Please DM us your order number so we can check the delivery status for you."

The key principle:

The response should be grounded in historical brand behavior rather than invented from general knowledge.

15. LLM's Role

If we add an LLM, its job is **not** to decide company policies.

Its job is:

Historical examples
+
New customer message
↓
LLM
↓
Natural-language response

Example:

Historical:

"Please DM your order number."

New:

"Hey, my package still hasn't arrived."

LLM can produce:

"Sorry about the delay. Please DM us your

order number so we can check the status."

The LLM is basically **rewriting/combining grounded information** into a natural response.

16. Escalation Decision

This is the third major AI component.

The agent must answer:

Can I safely handle this automatically?

There are two possible outputs:

AUTO-HANDLE

or

ESCALATE TO HUMAN

17. When Do We Auto-Handle?

We can auto-handle when:

High intent confidence
+
Strong historical match
+
No major risk signals
+
Known type of problem

Example:

Customer:
"What are your support hours?"

Intent confidence:
96%

Historical similarity:
0.89

Risk:

Low

Decision:

AUTO-HANDLE

18. When Do We Escalate?

We escalate when the AI shouldn't be trusted to handle the case automatically.

Examples:

Low confidence

Intent confidence = 43%

Decision:

ESCALATE

Reason:

The system is uncertain about the customer's intent.

No historical precedent

Similarity = 0.31

Decision:

ESCALATE

Reason:

No sufficiently similar historical support case was found.

Legal issue

"I'm contacting my lawyer."

Decision:

ESCALATE

Reason:

Potential legal issue requires human review.

Repeated unresolved complaint

"I contacted you three times and nobody fixed this."

Decision:

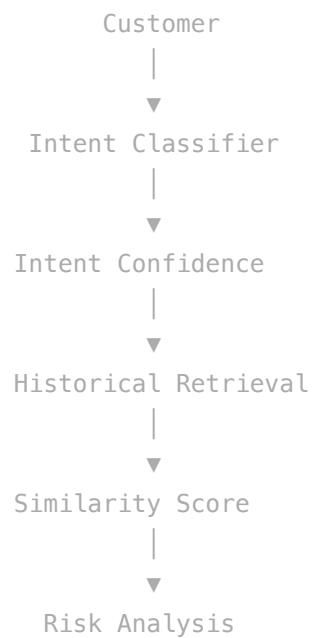
ESCALATE

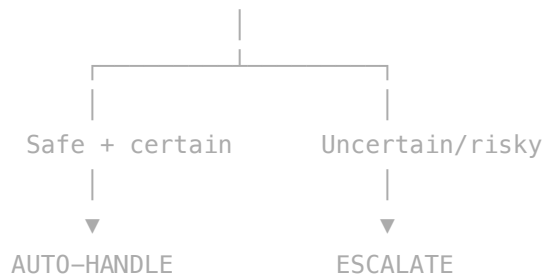
Reason:

Customer reports a repeated unresolved issue.

19. Final Decision Logic

Conceptually:





20. Why Escalation Is Important

Imagine:

Customer:
"I've been charged twice and this is the third
time I've contacted you."

The AI might understand the intent correctly:

```
payment_issue = 95%
```

But that doesn't necessarily mean it should auto-handle it.

The customer has an unresolved issue.

Therefore:

```
Intent confidence = HIGH  
BUT  
Risk = HIGH  
  
→ ESCALATE
```

This is a very important distinction.

Confidence in understanding $\neq$ confidence that the AI should act.

21. Every Escalation Needs a Reason

The assignment explicitly asks for:

"with a stated reason"

So don't output only:

ESCALATE

Instead:

Decision:
ESCALATE

Reason:
Customer reports a repeated unresolved billing issue
that may require account-specific investigation.

This makes the system explainable.

22. Complete Agent Output

For every incoming message, our system should ultimately produce:

```
{
  "customer_message": "My package hasn't arrived yet",
  "intent": "delivery_issue",
  "intent_confidence": 0.92,
  "historical_matches": [
    {
      "similarity": 0.91,
      "customer": "My package hasn't arrived",
      "brand_response": "Please DM us your order number."
    }
  ],
  "draft_response":
    "Sorry about the delay. Please DM us your order number so we can check the status.",
  "decision": "AUTO-HANDLE",
  "reason":
    "High-confidence intent with strong historical precedent and no major escalation."
}
```

This is essentially the **brain of your application**.

23. Evaluate the Entire Agent

Now we need to prove that it works.

We evaluate three separate things.

A. Intent

```
Accuracy
Precision
Recall
F1
```

B. Retrieval

```
Recall@1
Recall@3
Recall@5
```

Example:

```
Recall@1 = 78%
Recall@3 = 89%
Recall@5 = 94%
```

Meaning:

94% of relevant cases appeared within the top 5 retrieved historical examples.

C. Escalation

Measure:

```
Escalation precision
Escalation recall
Escalation F1
False Auto-Handle Rate
False Escalation Rate
```

24. Most Important Trust Metric

I would emphasize:

False Auto-Handle Rate

Suppose we manually identify:

100 cases that should go to a human

Your agent incorrectly handles:

4

Then:

False Auto-Handle Rate = 4%

This is powerful because it directly addresses:

"Is the system safe enough to trust?"

25. Human Evaluation of Replies

We should also manually evaluate a sample of generated responses.

For example, take:

100 generated replies

Have them evaluated for:

- Relevance
- Helpfulness
- Brand consistency
- Groundedness
- No hallucination

Example:

Metric	Result
Relevant	94%
Helpful	90%
Brand-consistent	93%
Grounded	96%

Metric	Result
No unsupported claims	97%

This gives you evidence that the agent isn't just producing fluent text.

26. Streamlit Application

Finally, we put everything into a user interface.

The user enters:

```
My package hasn't arrived yet.
```

The application displays:

INTENT

Delivery Issue

CONFIDENCE

92%

HISTORICAL CASES

Case 1

Similarity: 91%

Customer:

My package hasn't arrived.

Brand:

Please DM us your order number.

DRAFT RESPONSE

Sorry about the delay. Please DM us your order number so we can check the delivery status.

DECISION

 AUTO-HANDLE

Reason:
High confidence + strong historical precedent

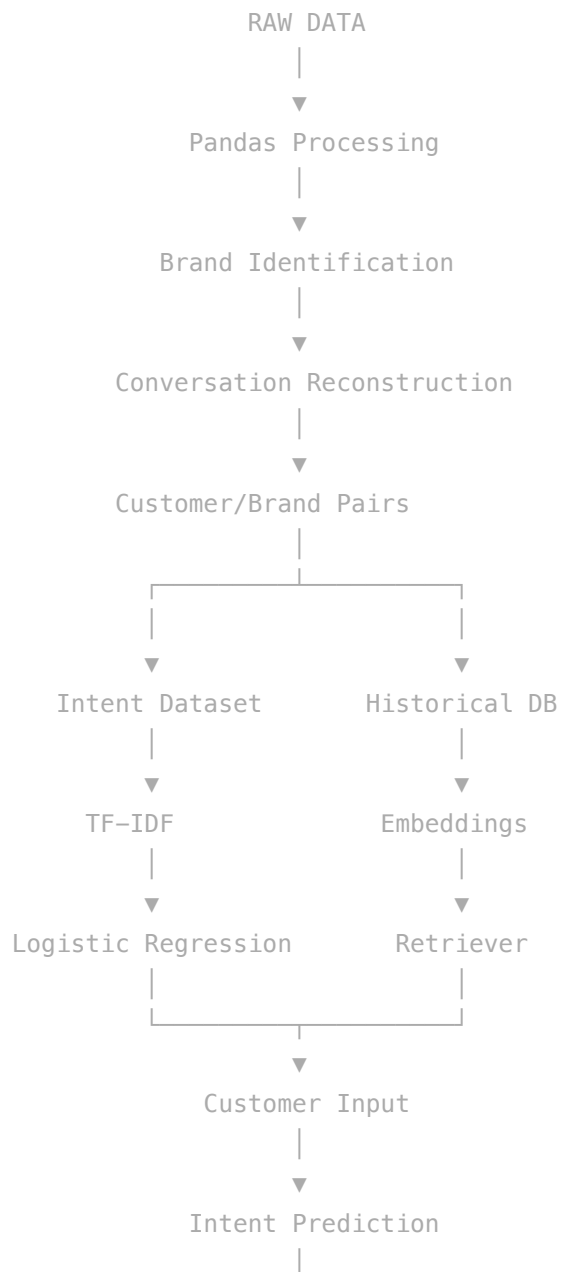
For a risky case:

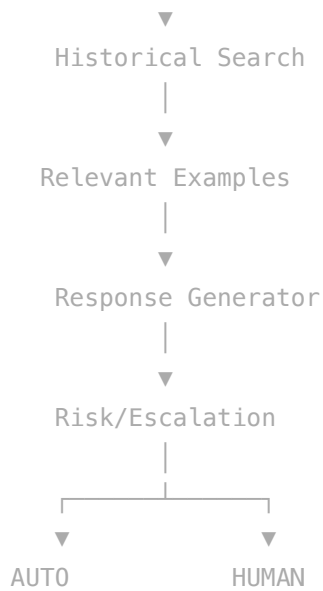
● ESCALATE TO HUMAN

Reason:
Repeated unresolved billing issue requiring
account-specific investigation.

27. Complete Technical Flow

From a developer perspective:





28. The Three Models/Systems

You can explain the project in an interview as three layers:

Layer 1 — Understanding

TF-IDF + Logistic Regression

Answers:

What does the customer want?

Layer 2 — Knowledge

Sentence Transformer
+
Semantic Search

Answers:

How did the brand handle similar cases before?

Layer 3 — Decision

Confidence
+

Similarity
+
Risk Rules

Answers:

Should the AI respond or should a human take over?

Then:

LLM

can be used to turn the retrieved historical information into a natural response.

29. What Makes This Project Strong

The important part is that you're **not claiming**:

"My AI can answer every customer."

Instead, you're saying:

"My agent knows when it has enough evidence to respond and when it should defer to a human."

That is a much stronger AI-agent design.

30. Final Project Story for Your Interview

If the interviewer asks:

"Explain your project."

You can say:

"I built an AI customer-support agent using the Customer Support on Twitter dataset. I selected one brand and analyzed its historical conversations to derive a brand-specific intent taxonomy. I trained an intent classifier to understand incoming customer requests and used sentence embeddings to retrieve semantically similar historical support cases. The response generator uses these historical cases as grounding so that the generated response follows the brand's previous support behavior. Finally, I implemented an escalation layer based on intent confidence, historical similarity, and risk signals. If the system doesn't have sufficient confidence or historical evidence, it escalates the case to a human with an explicit reason. I

evaluated intent classification, retrieval quality, response quality, and especially the false auto-handle rate to determine whether the agent is trustworthy."

That's the **core story of your project**.

31. The order we should actually implement it

Don't try to code everything at once.

Follow this exact order:

STEP 1

↓

Download dataset

STEP 2

↓

Load + inspect dataset

STEP 3

↓

Identify brands/support accounts

STEP 4

↓

Select one brand

STEP 5

↓

Reconstruct customer → brand conversations

STEP 6

↓

Analyze customer messages

STEP 7

↓

Define 6–10 intents

STEP 8

↓

Manually label training data

STEP 9

↓

Train intent classifier

STEP 10

↓

Evaluate classifier

STEP 11

↓

Build historical response database

STEP 12

↓

Generate sentence embeddings

STEP 13

↓

Build semantic retriever

STEP 14

↓

Evaluate retrieval

STEP 15

↓

Build response generator

STEP 16

↓

Build escalation engine

STEP 17

↓

Combine everything into Agent

STEP 18

↓

Evaluate complete agent

STEP 19

↓

Build Streamlit UI

STEP 20

↓

README + architecture diagram + report

STEP 21

↓

Final demo

The most important dependency is:

Dataset

↓

Brand

↓
Conversations
↓
Intents
↓
Classifier

So we should not start by writing the Streamlit application or LLM code. First we need to understand the actual dataset and select the brand. Once that foundation is correct, the remaining components become much easier and more defensible.



also include which models used

Yes. We should include the **models/algorithms used at each stage** in the project flow. This makes the architecture much clearer for your evaluator.

Complete Project Flow with Models Used

```
Customer Support on Twitter Dataset
↓
1. Data Preprocessing
↓
2. Select One Brand
↓
3. Extract Conversations
↓
4. Analyze Customer Problems
↓
5. Define Intent Categories
↓
6. Manually Label Training Data
↓
7. Intent Classification
TF-IDF + Logistic Regression
↓
8. Intent Confidence
predict_proba()
↓
9. Historical Knowledge Base
↓
10. Text Embeddings
Sentence Transformer
all-MiniLM-L6-v2
↓
```

```
11. Similarity Retrieval
Cosine Similarity
    ↓
12. Response Generation
Historical Response / LLM
    ↓
13. Trust & Risk Decision
Rule-based Escalation
    ↓
14. Final Decision
AUTO-HANDLE / ESCALATE
    ↓
15. Streamlit Application
```

1. Data Preprocessing

Tools:

- Python
- Pandas
- NumPy

We load the Twitter dataset and clean the text.

Example:

```
"My package hasn't arrived!!!"
    ↓
"My package hasn't arrived!"
```

We also identify:

- Customer tweets → `inbound = True`
- Brand/support tweets → `inbound = False`

2. Brand Selection

We use **Pandas aggregation**, not an ML model.

For example:

```
df[df["inbound"] == False]["author_id"].value_counts()
```

This tells us which support account has the largest number of responses.

3. Conversation Extraction

Again, no ML model is required here.

We use:

- `tweet_id`
- `in_response_to_tweet_id`
- `response_tweet_id`

to connect:

```
Customer message
      ↓
Brand response
```

Example:

```
Customer:
"My package hasn't arrived."

      ↓

Brand:
"Sorry about the delay. Please send us your order number..."
```

This pair becomes our **historical support example**.

4. Intent Classification

This is our first major ML model.

Model used:

TF-IDF + Logistic Regression

```
Customer Message
      ↓
TF-IDF Vectorizer
      ↓
Numerical representation
      ↓
Logistic Regression
      ↓
Predicted Intent
```

Example:

"My package is still not delivered"

↓

TF-IDF Vectorizer

↓

Logistic Regression

↓

delivery_issue

Why Logistic Regression?

Because:

- Fast
- Easy to train
- Works well for text classification
- Gives class probabilities
- Easy to explain to evaluators
- Strong baseline for a relatively small manually labeled dataset

5. TF-IDF

TF-IDF itself is **not a model**. It is a **text feature extraction technique**.

It converts text into numbers.

For example:

"My package hasn't arrived"

becomes something like:

[0.00, 0.42, 0.31, 0.17, ...]

These numerical features are given to Logistic Regression.

6. Intent Confidence

After classification, Logistic Regression gives probabilities.

Example:

delivery_issue	0.91
refund_request	0.04

payment_issue	0.03
account_issue	0.02

Therefore:

```
Predicted intent = delivery_issue
Confidence = 91%
```

We use:

```
model.predict_proba()
```

Important point for your presentation

Confidence is not the same as trust.

For example:

```
Intent confidence = 95%
Historical similarity = 42%
```

The model understands the customer's problem, but we don't have a strong historical example.

Therefore:

```
ESCALATE
```

This is an important part of making your system trustworthy.

7. Historical Knowledge Retrieval

For this part we use another model.

Model:

Sentence Transformer

Specifically:

```
sentence-transformers/all-MiniLM-L6-v2
```

This is an **embedding model**.

It converts sentences into semantic vectors.

Example:

```
"My package hasn't arrived"
  ↓
Sentence Transformer
  ↓
[0.21, -0.13, 0.72, ...]
```

Another sentence:

```
"My delivery is missing"
  ↓
Sentence Transformer
  ↓
[0.19, -0.10, 0.69, ...]
```

Their vectors will be close because they have similar meanings.

8. Cosine Similarity

Cosine similarity is **not a machine-learning model**.

It is the mathematical method used to compare the embeddings.

```
New customer message
  ↓
MiniLM embedding
  ↓
Cosine similarity
  ↓
Historical customer messages
```

Example:

Historical Customer Message	Similarity
My package hasn't arrived	0.92
Where is my delivery?	0.88
I need a refund	0.31
My card was charged twice	0.18

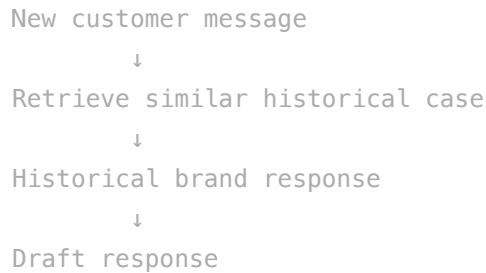
Therefore, we retrieve the top historical cases.

9. Response Generation

There are **two possible versions**.

Version 1 — Safe baseline

Use the response from the most similar historical case.

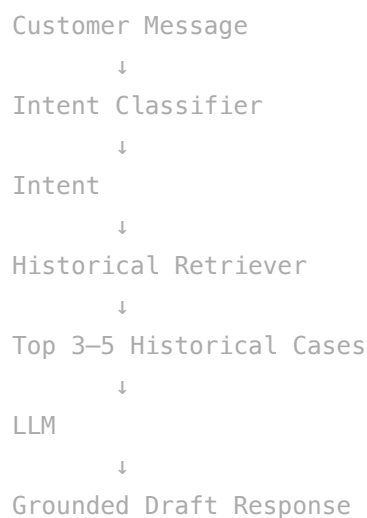


This is very safe because the response is directly grounded in the brand's historical behavior.

Version 2 — Better final system

Use an **LLM** to generate a response using retrieved historical examples.

Architecture:



The LLM receives:

Customer message
Predicted intent

Historical brand responses

and generates a new response.

Important rule

The LLM should **not** invent:

- Refund policies
- Prices
- Delivery dates
- Account information
- Compensation
- Company policies

It should only use information supported by the retrieved historical examples.

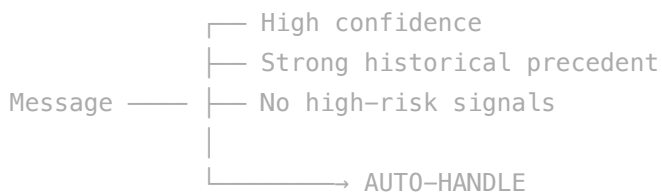
10. Escalation Model

For the first version, I recommend a **rule-based risk engine**, rather than another ML model.

Inputs:

Intent confidence
+
Historical similarity
+
Risk signals
+
Repeated complaint
+
Account-specific issue

Then:



Otherwise:

Message

↓

Risk detected / insufficient evidence

↓

ESCALATE TO HUMAN

11. Example

Suppose customer sends:

"My package hasn't arrived yet. Can you help?"

Step 1 — Intent model

TF-IDF + Logistic Regression

```
delivery_issue = 94%
refund_request = 3%
complaint      = 2%
other          = 1%
```

Prediction:

```
Intent = delivery_issue
Confidence = 94%
```

Step 2 — Retrieval

all-MiniLM-L6-v2

Finds:

```
"My delivery hasn't arrived"
Similarity = 91%
```

Step 3 — Response

Historical response:

"Sorry about the delay. Please send us your order number so we can check this for you."

The system can use this directly or give it to an LLM as grounding.

Step 4 — Escalation

Intent confidence = 94%

Similarity = 91%

Risk = Low

Therefore:

● AUTO-HANDLE

Reason:

High-confidence intent classification and strong historical precedent with no major risk signals.

12. Example Where It Escalates

Customer:

"I've contacted you three times and nobody has fixed this. I'm going to take legal action."

Intent model:

complaint = 88%

But the escalation engine detects:

Repeated unresolved issue
+
Legal threat

Therefore:

● ESCALATE TO HUMAN

Reason:

Customer reports a repeated unresolved issue and mentions potential legal action.

This demonstrates **trustworthiness**.

13. Models/Techniques Used — Final Table

Component	Model / Technique	Purpose
Data processing	Pandas	Clean and process dataset
Text preprocessing	Python	Clean customer messages
Intent features	TF-IDF	Convert text to numerical features
Intent classifier	Logistic Regression	Predict customer intent
Intent confidence	<code>predict_proba()</code>	Measure classification confidence
Semantic embeddings	all-MiniLM-L6-v2	Convert text into semantic vectors
Retrieval	Cosine Similarity	Find similar historical cases
Response	Historical response / LLM	Generate grounded reply
Risk decision	Rule-based engine	Decide auto-handle vs escalation
UI	Streamlit	Build interactive application

So the main AI/ML models are:

1. Logistic Regression

→ Intent classification

2. Sentence Transformer (all-MiniLM-L6-v2)

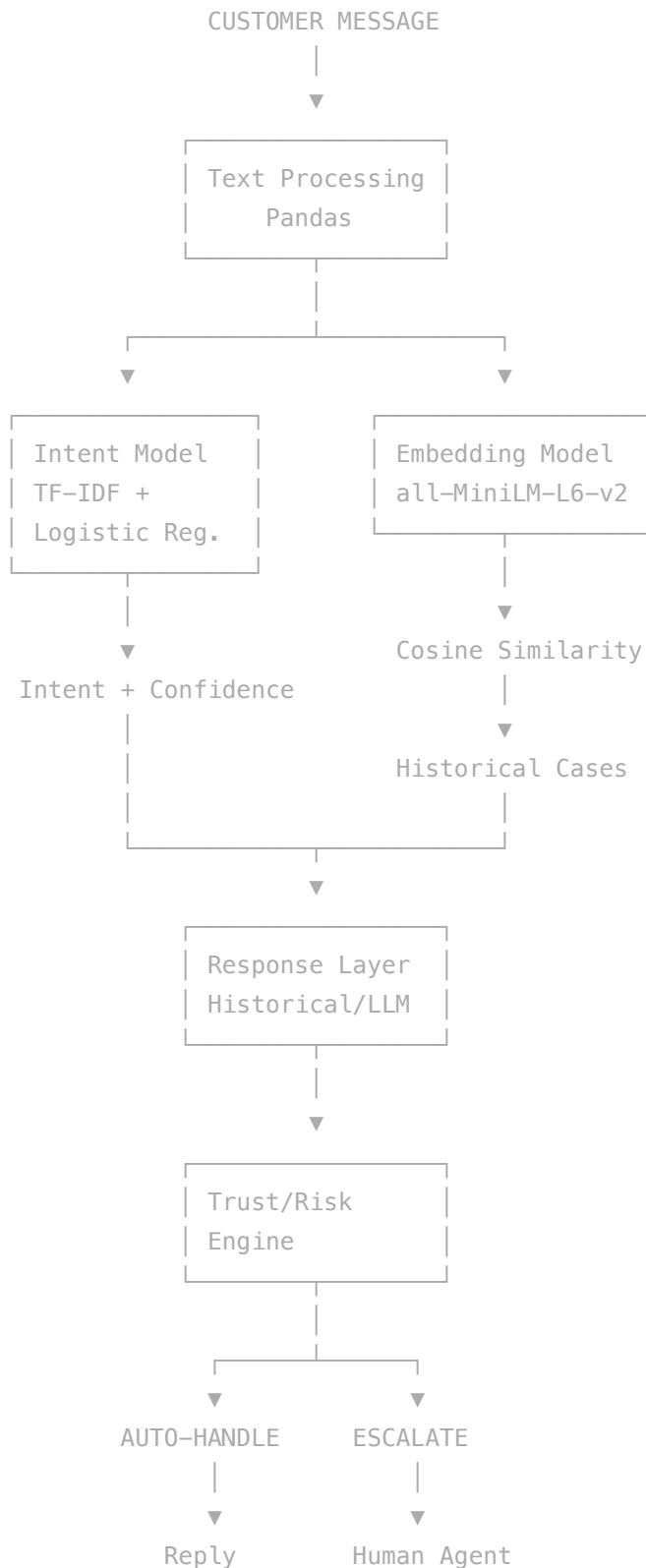
→ Semantic embeddings and historical-case retrieval

3. Optional LLM

→ Grounded response generation

The **rule-based escalation engine is not an ML model**; it is a safety/trust layer.

Overall architecture to show the evaluator



The strongest story for your assignment is:

"The system does not simply generate a response. It first understands the intent, retrieves historical evidence, grounds the response in that evidence, and independently checks whether it is safe to handle automatically. If the evidence or confidence is insufficient, it escalates to a human."

ChatGPT can make mistakes. Check important info.